

# Гайдлайн проекта: семантический поиск по научным статьям

Deep Learning for Search, Summer 2026 · Innopolis University · Сегодня: 3 июля ·  
Защита проекта: среда, 15 июля (Week 7), презентация 15 мин + 5 мин вопросы.

## Коротко о проекте

Мы делаем поисковик по научным статьям: пользователь вводит запрос обычным языком (например, «методы обнаружения поддельных изображений»), а система находит релевантные статьи, даже если в них используются другие слова. Это решает проблему обычного поиска по ключевым словам, который такие статьи пропускает.

Ключевые решения проекта (чтобы все понимали одну картину):

- Датасет: arXiv (Kaggle, Cornell-University/arxiv). Берём статьи по категориям cs.CL, cs.LG, cs.AI, cs.CV, cs.IR, stat.ML — цель 600 000–700 000 статей (с запасом над требованием в 500 000).
- Итерация 1 (baseline, неделя 1): модель эмбедингов all-MiniLM-L6-v2 → простой (Flat) индекс в Qdrant → поиск по косинусному расстоянию.
- Итерация 2 (улучшенная, неделя 2): доменная модель SPECTER2 + квантизация (int8) → индекс HNSW в Qdrant → гибридный поиск (BM25 + dense-поиск, слияние через RRF).
- Сервис: FastAPI-бэкенд + простая веб-страница поиска, всё поднимается через Docker.
- Оценка: ручной валидационный набор из 80–100 запросов, метрики nDCG@10, Recall@10, MAP, задержка ответа, размер индекса и модели.

## Общие правила — чтобы не было проблем

- Один репозиторий на всех (GitHub): папки /data, /embeddings, /index, /search, /service, /docs. Большие файлы (датасет, векторы) — в общей папке Google Drive, не в гите.
- Единый формат данных: у каждой статьи одно и то же поле-идентификатор paper\_id везде — в датасете, в векторах, в индексе, в ответах поиска. Тексты — UTF-8, векторы — float32. Это самая частая причина проблем при сборке, проверьте это в первую очередь.
- Порядок передачи данных: Данные → Эмбединги → Хранилище/индекс → Поиск/оценка → Сервис. Каждый не ждёт полный датасет — как только Данные выложат маленькую подвыборку (1 000–5 000 статей), остальные уже могут писать и тестировать код на ней.
- Синхронизация: короткое сообщение в чат каждый день (или через день) — что сделано, что блокирует, что нужно от других.
- Главное правило недели 1: к концу недели 1 весь путь запрос → ответ должен работать целиком, пусть даже грубо. Неделя 2 — это уже улучшение, а не latest-minute сборка с нуля.

## Таймлайн

| Период            | Что происходит                                                                                                                              | Цель                           |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|
| Неделя 1 3–9 июля | Сбор и чистка данных, первая модель эмбедингов, простой индекс, базовый поиск, скелет сервиса. 8 июля — обычная пара (лекция + дедлайн АЗ). | Итерация 1 работает end-to-end |

|                        |                                                                                                                                                                                      |                                 |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| Неделя 2<br>10–15 июля | 10 июля — финальный экзамен (весь день, меньше времени на проект в этот день). Улучшенная модель, HNSW-индекс, гибридный поиск, сборка сервиса, слайды, репетиция. 15 июля — защита. | Итерация 2 + сравнение + защита |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|

## Задачи по ролям

### 1. Данные

Сбор, чистка и обоснование датасета + валидационный набор запросов

#### Неделя 1 (3 – 9 июля) — цель: готовый чистый датасет и первая версия валидационного набора

- День 1–2: скачать датасет arXiv с Kaggle, изучить категории и годы, выбрать срез (категории + период) так, чтобы получилось 600–700 тыс. статей.
- День 3: почистить тексты (убрать LaTeX-мусор, пустые abstract, дубликаты), сохранить в один файл (Parquet) с колонками paper\_id, title, abstract, category, year.
- День 4: сравнить распределение категорий и годов до и после среза (пара графиков) — это и есть обоснование, что срез представляет весь датасет, требуемое по заданию.
- День 5: выложить файл в общую папку, сообщить команде. Начать валидационный набор: 40–50 запросов + вручную найденные релевантные статьи (каждый в команде добавляет 8–10 своих запросов).
- День 6–7: свести валидационный набор в один файл (query, relevant\_paper\_ids), помогать другим по мере надобности.

Сдать к концу недели 1: arxiv\_clean.parquet (600–700 тыс. статей) + validation\_set\_v1.csv (40–50 запросов)

Передать дальше: файл данных → Эмбединги (роль 2); валидационный набор → Поиск и оценка (роль 4)

#### Неделя 2 (10 – 15 июля) — цель: расширенный валидационный набор и раздел слайдов про датасет

- День 8–9: расширить валидационный набор до 80–100 запросов, добавить «смысловые» запросы без прямого совпадения слов — на них будет видна разница между обычным и смысловым поиском.
- День 10: вручную проверить 15–20 случайных результатов поиска итерации 2 — выглядят ли они релевантными (быстрая проверка на здравый смысл).
- День 11: подготовить слайд(ы) — почему выбран этот датасет, как обоснован срез, в чём ценность проекта и для кого он.
- День 12: финальная проверка данных перед демо — нет ли битых записей, совпадают ли id между датасетом и индексом.
- День 13 (защита): готов отвечать на вопросы про датасет и обоснование среза.

Сдать к концу недели 2: validation\_set\_final.csv (80–100 запросов) + слайд про датасет

Передать дальше: финальный валидационный набор → Поиск и оценка (роль 4)

### 2. Модель представления (эмбединги)

Выбор модели, кодирование всех статей в вектора, анализ и сжатие модели

#### Неделя 1 (3 – 9 июля) — цель: все статьи закодированы в вектора первой моделью

- День 1-2: пока нет полного датасета — поднять sentence-transformers, выбрать модель all-MiniLM-L6-v2 (быстрая, 384 измерения, хорошо изучена — обосновать это как выбор для baseline), потренироваться на маленькой подвыборке.
- День 3: как только придёт чистый датасет от роли 1 — написать скрипт кодирования пачками (батчами по 256-512 текстов, желательно на GPU).
- День 4: прогнать модель на всех 600-700 тыс. статей, сохранить вектора в формате, который примет Qdrant (например .npy + таблица id → индекс вектора).
- День 5: передать вектора и модель роли 3 (хранилище и индексация).
- День 6-7: начать анализ векторов — PCA, сколько компонент объясняют большую часть разброса (это нужно для обоснования сжатия модели во второй итерации). Короткий отчёт на полстраницы.

Сдать к концу недели 1: embeddings\_v1.npy (вектора для всех статей) + отчёт по PCA-анализу

*Передать дальше: вектора и код кодирования → Хранилище и индексация (роль 3)*

## **Неделя 2 (10 - 15 июля) — цель: улучшенная модель (доменная + квантизация) и сравнение с итерацией 1**

- День 8-9: подключить SPECTER2 — модель, обученную именно на научных статьях, прогнать на том же датасете, сохранить embeddings\_v2.
- День 10: применить int8-квантизацию к векторам/модели, замерить во сколько раз уменьшился размер и как изменилась скорость, сильно ли просела точность.
- День 11: вместе с ролью 4 свести таблицу сравнения моделей: размер вектора, размер на диске, время кодирования всего датасета, точность на валидации.
- День 12: подготовить слайд «модель и её анализ» — PCA, квантизация, почему выбраны именно эти модели.
- День 13: готов отвечать на вопросы «почему эта модель, а не другая».

Сдать к концу недели 2: embeddings\_v2.npy (квантизованные, доменные) + таблица сравнения моделей

*Передать дальше: вектора v2 → Хранилище и индексация (роль 3); таблица → Сервис/презентация (роль 5)*

## **3. Хранилище и индексация**

Векторная база данных (Qdrant) и алгоритмы индексации

## **Неделя 1 (3 - 9 июля) — цель: Qdrant поднят, простой индекс работает, поиск по вектору отвечает**

- День 1-2: поднять Qdrant в Docker, разобраться в API (создание коллекции, загрузка векторов, поиск по вектору).
- День 3: пока нет реальных векторов — проверить весь пайплайн на случайных тестовых векторах, чтобы код был готов заранее.
- День 4-5: как только придут вектора от роли 2 — загрузить их в коллекцию papers\_v1, построить простой индекс (Flat или HNSW с настройками по умолчанию).
- День 6: замерить время построения индекса, объём занятой памяти, задержку одного запроса.
- День 7: дать роли 4 рабочий доступ к коллекции для тестов, кратко описать как подключаться (адрес, имя коллекции, пример запроса).

Сдать к концу недели 1: коллекция papers\_v1 в Qdrant + короткая инструкция подключения

*Передать дальше: доступ к коллекции → Поиск и оценка (роль 4)*

## Неделя 2 (10 - 15 июля) — цель: улучшенный индекс HNSW и сравнение подходов индексации

- День 8-9: создать коллекцию papers\_v2 под новые вектора (роль 2), настроить HNSW (параметры ef\_construct, m, ef\_search), сравнить с Flat по скорости и памяти на одинаковых запросах.
- День 10: попробовать разные значения ef\_search — компромисс между скоростью и точностью, зафиксировать лучший вариант.
- День 11: свести таблицу: индекс / время построения / память / средняя задержка — для итогового сравнительного исследования.
- День 12: подготовить слайд «индексация: почему HNSW, а не Flat» + оценку по памяти для hardware requirements.
- День 13: готов ответить на вопросы про индексацию и требования к «железу» для хранения индекса.

Сдать к концу недели 2: коллекция papers\_v2 (HNSW) + таблица сравнения индексов

Передать дальше: итоговые числа по памяти/скорости → Сервис и презентация (роль 5)

## 4. Поиск и оценка

Пайплайн поиска, метрики качества, сравнение итераций

## Неделя 1 (3 - 9 июля) — цель: рабочий пайплайн поиска и первые числа метрик по итерации 1

- День 1-3: пока нет доступа к индексу — написать код метрик (nDCG@10, Recall@10, MAP, замер задержки ответа) и проверить его на игрушечном примере, чтобы не терять время потом.
- День 4-5: как только будет доступ к индексу (роль 3) и валидационный набор (роль 1) — собрать пайплайн: текст запроса → вектор → поиск в Qdrant → топ-10 статей.
- День 6: прогнать все запросы из валидационного набора через пайплайн итерации 1, посчитать метрики.
- День 7: свести результаты в таблицу, передать роли 5 готовую функцию поиска (простой Python-модуль, который принимает текст запроса и возвращает список статей).

Сдать к концу недели 1: функция search(query) → топ-10 статей + таблица метрик итерации 1

Передать дальше: функция поиска → Сервис и презентация (роль 5)

## Неделя 2 (10 - 15 июля) — цель: гибридный поиск (BM25 + dense через RRF) и сравнительное исследование

- День 8-9: добавить BM25-поиск по тем же текстам (например, через rank\_bm25), реализовать слияние результатов BM25 и dense-поиска через RRF.
- День 10 (если есть время): добавить cross-encoder reranking для топ-20 результатов ради ещё большей точности.
- День 11: прогнать валидационный набор через итерацию 2, посчитать те же метрики, сравнить с итерацией 1 в одной таблице — это и есть требуемое сравнительное исследование по заданию.
- День 12: подготовить слайд «поиск и оценка» + чёткий вывод, какой подход эффективнее и почему (обязательный вопрос на защите).
- День 13: готов ответить на вопросы про метрики и пайплайн экспериментов.

Сдать к концу недели 2: функция hybrid\_search(query) + итоговая таблица сравнения итерация 1 vs 2

## 5. Сервис и презентация

API, демо-интерфейс, сборка слайдов и защита

### Неделя 1 (3 - 9 июля) — цель: скелет сервиса поднят и работает локально хотя бы с заглушкой поиска

- День 1-2: поднять проект на FastAPI, эндпоинт /search (принимает текст запроса, возвращает список статей-заглушек) и /health.
- День 3-4: настроить Docker (Dockerfile + docker-compose вместе с Qdrant), чтобы весь сервис поднимался одной командой.
- День 5: как только роль 4 передаст функцию поиска — подключить её в /search вместо заглушки.
- День 6: сделать простую веб-страницу — поле ввода запроса и список результатов (обычный HTML+JS или Streamlit, без излишеств).
- День 7: проверить весь путь целиком: открыть страницу → ввести запрос → увидеть реальные результаты итерации 1.

Сдать к концу недели 1: работающий локально сервис (Docker) с итерацией 1 и простым интерфейсом

Передать дальше: —

### Неделя 2 (10 - 15 июля) — цель: сервис с обеими итерациями, готовые слайды, отрепетированная защита

- День 8-9: подключить итерацию 2 (гибридный поиск), сделать переключатель «итерация 1 / итерация 2» в интерфейсе — это удобно показать разницу на защите.
- День 10: собрать финальные цифры по hardware requirements (сколько памяти и видеокарты нужно для кодирования, сколько для инференса) — взять у ролей 2 и 3.
- День 11: собрать слайды всех участников в одну презентацию (14-15 слайдов на 15 минут), проверить, что все 6 обязательных вопросов (см. чек-лист ниже) покрыты явно.
- День 12: командная репетиция с таймером (уложиться в 15 минут) и финальная проверка, что демо не падает и не виснет.
- День 13 (защита): показывает демо и координирует ответы на вопросы комиссии.

Сдать к концу недели 2: финальный сервис + готовая презентация + отрепетированное демо

Передать дальше: —

## Чек-лист перед защитой

Комиссия обязательно спросит про эти 6 пунктов — за каждым закреплён ответственный, но на защите отвечать может любой из команды.

| Обязательный вопрос                                         | Кто готовит ответ                   |
|-------------------------------------------------------------|-------------------------------------|
| В чём ценность проекта и кому он нужен?                     | Данные + Сервис/презентация         |
| Какая общая архитектура системы?                            | Сервис/презентация (сводит от всех) |
| Какие ключевые технологии использованы и почему именно они? | Каждый — по своей части             |

|                                                              |                                                    |
|--------------------------------------------------------------|----------------------------------------------------|
| Какой экспериментальный пайплайн и как оценивалось качество? | Поиск и оценка                                     |
| Какой подход эффективнее и почему?                           | Поиск и оценка + Модель представления + Индексация |
| Какие требования к «железу» для разработки и инференса?      | Модель представления + Индексация + Сервис         |

*Напоминание: датасет должен содержать не меньше 500 000 объектов (у нас с запасом — 600–700 тыс.), срез должен быть обоснован, а всё сравнение итераций 1 и 2 нужно логировать по ходу работы — не восстанавливать в последний день.*